

Поиск хайлайтов в видео: промежуточный отчёт этапа 1

Дата подготовки: 7 апреля 2026

1. Постановка задачи, требования и ограничения

Цель проекта - построить систему временной локализации хайлайтов в видео. На вход подаётся видео V длительностью T , на выходе ожидается набор временных интервалов $[t_s, t_e]$ или поклиповая функция важности s_t , по которой после пороговой обработки формируются интервалы хайлайтов. В базовой постановке я рассматриваю query-independent detection: система должна найти наиболее интересные или вовлекающие моменты без текстового запроса пользователя. Это ближе всего к сценариям автонарезки, превью, "лучшие моменты" для пользовательского видео и массовой модерации/индексации.

Дополнительно полезен query-conditioned вариант: пользователь формулирует запрос, например "map unpacks groceries", а модель находит релевантные моменты и самые удачные фрагменты внутри них. Такой режим покрывается QVHighlights и Moment-DETR, но для проекта этапа 2 я считаю его расширением, потому что критерии задачи сформулированы как общий поиск хайлайтов в видео, без обязательного natural language query.

Формально это задача temporal detection/ranking. Возможны три эквивалентных представления таргета:

- бинарная классификация клипов фиксированной длины: highlight / non-highlight;
- регрессия или ранжирование непрерывного score для каждого кадра/клипа;
- прямое предсказание интервалов, где качество зависит от temporal IoU с разметкой.

Ключевые требования:

- Точность локализации: интервалы не должны быть слишком длинными и не должны покрывать почти всё видео. Для оценки нужны метрики с учётом temporal IoU и/или MAP по клипам.
- Масштабируемость: система должна обрабатывать много видео; поэтому предпочтительна схема "feature extractor -> лёгкая temporal model" с предрассчитанными эмбедингами. Полный end-to-end video transformer дороже и требует больше GPU.
- Устойчивость к субъективности: хайлайт - это не просто summary. В Mr. HiSum отдельно подчёркнуто, что summary-score отвечает за представление сюжета, а highlight-score - за интересность момента. Поэтому нужно фиксировать определение таргета для каждого датасета.
- Ограничения данных: YouTube-источники могут исчезать, метки субъективны, длительности и жанры различаются, а в некоторых датасетах доступен только proxy-label, например Most replayed.

- Ограничения внедрения: для массового применения желательно декодировать видео с низкой частотой, например 1-3 fps или 2-секундные клипы, и хранить признаки отдельно от сырого видео.

2. Данные: источники, лицензии, структура, объём

2.1 TVSum

TVSum50 - классический малый датасет для video summarization, часто используемый и для highlight detection после преобразования importance scores в бинарные клипы. По статье TVSum содержит 50 YouTube-видео из 10 категорий TRECVID MED, по 5 видео на категорию, длительностью 2-10 минут, с Creative Commons-видео. Полный объём в таблице авторов - 50 видео, 352 356 кадров, 3ч 29м 42с. Разметка: 1 000 ответов Amazon Mechanical Turk, то есть 20 оценщиков на видео. Каждый оценивал равномерные 2-секундные шоты по шкале 1-5; звук был выключен, чтобы оценка опиралась на визуальные признаки.

Плюсы: реальные видео, ручные оценки, высокая пригодность для быстрой проверки baseline. Минусы: всего 50 видео, сильная дисперсия результатов на train/test split, не является изначально pure highlight detection. Для этапа 2 TVSum можно использовать как sanity-check и внешний тест, но не как основной источник обучения.

Лицензия и ограничения: видео отбирались под Creative Commons, но при фактической загрузке нужно сохранять ссылки/атрибуцию и проверять доступность конкретных YouTube-роликов.

2.2 QVHighlights

QVHighlights - query-based датасет для moment retrieval и highlight detection. Он содержит 10 310 queries, 18 367 моментов и 10 148 видео из трёх доменов: daily vlog, travel vlog, news. Сырые видео нарезаны на 150-секундные клипы. Разметка: свободный текстовый запрос, один или несколько релевантных временных интервалов, а также 5-балльные saliency scores для каждого query-relevant 2-секундного клипа. Для saliency используется 3 независимых оценщика; 4 означает Very Good, 0 - Very Bad. Test split не содержит закрытые поля `relevant\_clip\_ids`, `relevant\_windows`, `saliency\_scores`.

Структура JSONL-строки:

```
{
  "qid": 9769,
  "query": "...",
  "duration": 150,
  "vid": "youtubeid_start_end",
  "relevant_windows": [[72, 82], [84, 94]],
  "relevant_clip_ids": [36, 37, 38],
  "saliency_scores": [[4, 3, 2], [4, 1, 3], [4, 3, 4]]
}
```


Плюсы: готовый train/val/test, удобные JSONL-аннотации, высокая детализация интервалов, пригоден для метрик temporal detection. Минусы: это query-conditioned задача, поэтому для query-independent продукта нужен либо отдельный режим, либо фиктивный/общий query, что может сместить постановку.

Лицензия и ограничения: код Moment-DETR на GitHub указан как MIT; raw-видео загружаются с YouTube, поэтому нужно соблюдать условия YouTube. Для этапа 1 в репозитории использованы только публичные JSONL-аннотации.

2.3 Mr. HiSum

Mr. HiSum - крупный датасет для highlight detection и summarization. По статье он содержит 31 892 видео, 1 788 часов, 3 509 категорий и агрегированные сигналы от 50 000+ зрителей на видео. Источник метки - YouTube Most replayed: последовательность из 100 нормированных score от 0 до 1, каждый score соответствует равномерному сегменту видео. Авторы используют YouTube-8M frame-level visual features на 1 fps; видео длиннее 300 секунд и музыкальная категория исключаются. Длительность оставшихся видео - 121-300 секунд, средняя 201.9 секунды.

Структура `mr\_hisum.h5` по README:

- `features`: frame-level признаки YouTube-8M;
- `gtscore`: нормированные Most replayed scores;
- `change\_points`: shot boundaries через Kernel Temporal Segmentation;
- `gtsummary`: summary после 0/1 knapsack по шотам.

Плюсы: масштаб, широкое покрытие, прямой проху под "интересность/вовлечение", устойчивее evaluation, чем 50-видео TVSum. Минусы: метки не ручные, а поведенческий проху; bias зависит от интерфейса YouTube, аудитории и просмотров; нужны YouTube-8M features и отдельная сборка `features`.

Лицензия: README MR.HiSum указывает CC BY 4.0 для датасета и необходимость соблюдения YouTube Terms of Service и YouTube API Services Terms.

3. EDA и проверка качества данных

Для фактического анализа на этапе 1 я скачал только JSONL-аннотации QVHighlights, так как они небольшие и не требуют загрузки видео. Скрипт `scripts/qvhighlights\_eda.py` и notebook `notebooks/qvhighlights\_eda.ipynb` строят статистики и проверки валидности.

Основные результаты:

```
split | rows | unique videos | total hours | median duration | mean windows | median window length | mean  
relevant coverage | issues  
train | 7218 | 7100 | 299.84 | 150 s | 1.77 | 16 s | 0.298 | 0  
val | 1550 | 1519 | 64.42 | 150 s | 1.81 | 16 s | 0.281 | 0  
test | 1542 | 1529 | 64.06 | 150 s | n/a | n/a | n/a | 0
```

Проверки качества:

- во всех train/val примерах ``0 <= start < end <= duration``;
- длина ``saliency_scores`` совпадает с ``relevant_clip_ids``;
- каждый saliency score содержит ровно 3 оценки;
- в train/val нет пустых ``relevant_clip_ids``;
- test ожидаемо не содержит закрытые поля разметки.

Наблюдения по распределениям:

- почти все клипы имеют длительность 150 секунд, минимальная длительность 120 секунд;
- медианное число релевантных окон на видео равно 1, но встречаются примеры до 24-25 окон;
- медианная длина окна 16 секунд, есть короткие 2-секундные моменты и длинные до всего видео;
- средняя доля релевантного времени в train около 29.8%, в val около 28.1%;
- saliency-оценки сдвинуты к 3-4, потому что они ставятся только для релевантных клипов; это ожидаемо и не означает баланс между highlight/non-highlight по всему видео;
- медианный разброс между тремя аннотаторами равен 1, но максимум равен 4, что показывает субъективность "хайлайтности".

Вывод для этапа 2: QVHighlights подходит для отладки метрик и pipeline, но если основной режим query-independent, нужно либо выбрать Mr. HiSum как основной train, либо явно заявить query-conditioned вариант продукта.

[missing image: /Users/azamath/Desktop/vk_maga/deep_cv/outputs/qvhighlights_eda/split_sizes.png]

[missing image:

/Users/azamath/Desktop/vk_maga/deep_cv/outputs/qvhighlights_eda/duration_window_coverage.png]

[missing image: /Users/azamath/Desktop/vk_maga/deep_cv/outputs/qvhighlights_eda/saliency_scores.png]

4. Представления и признаки

Для этапа 2 рационально начать с предобученных признаков:

- визуальные frame/clip embeddings: CLIP ViT-B/32, SlowFast, VideoMAE или YouTube-8M Inception-v3 features;
- аудио как опциональный сигнал: полезно для спорта, концертов, реакций аудитории, но усложняет сборку и лицензии;
- текстовый query embedding для QVHighlights: CLIP text encoder или аналогичный text encoder;
- temporal features: позиционные признаки, длительность, соседние окна, локальные сглаживания score.

Причина выбора feature-based подхода: он дешевле для домашнего проекта и лучше масштабируется. End-to-end видео transformer по сырым кадрам потребует больше GPU и данных, а в критериях этапа 1 важна не максимальная SOTA, а корректная постановка и воспроизводимый baseline.

5. Подходы и baseline

Baseline 0: эвристика по равномерному или случайному выбору top-k клипов. Нужна только для контроля метрик. Если модель хуже случайной/равномерной, pipeline или метрика ошибочны.

Baseline 1: clip-level binary/ranking classifier. Нарезаем видео на 2-5 секундные клипы, извлекаем признаки, обучаем MLP/Logistic Regression/LightGBM на score. Для TVSum и Mr. HiSum можно использовать top-15% или top-50% как positive labels. Для QVHighlights положительный класс - релевантные клипы с saliency $\geq$ threshold, отрицательный - нерелевантные клипы. На инференсе score сглаживается, затем применяется threshold и merge соседних интервалов.

Baseline 2: temporal sequence model поверх embeddings. Используем BiLSTM/TCN/Transformer encoder, который получает последовательность clip embeddings и выдаёт score на каждый клип. Это лучше учитывает контекст, чем независимый MLP, и остаётся дешевле полного video transformer.

Baseline 3: Moment-DETR для QVHighlights. Он напрямую предсказывает moment coordinates и saliency scores из video+query features, используя transformer encoder-decoder и set prediction. Подходит, если в проекте нужен query-conditioned поиск, но для query-independent режима будет избыточен.

Baseline 4: модели из Mr. HiSum: PGL-SUM, VASNet, SL-module. Их стоит использовать как ориентир для обучения на Mr. HiSum, потому что авторы уже сравнивают F1 и MAP на этом датасете.

Мой выбор для этапа 2: основной путь - feature extractor + temporal model с поклипковым score и постобработкой в интервалы. Причины:

- метки Mr. HiSum и TVSum естественно являются score на кадр/шот;
- QVHighlights легко сводится к clip scoring;
- можно честно сравнить качество/скорость/ресурсы;
- модель проще обучать и отлаживать;
- threshold, NMS/merge и минимальная длительность интервала контролируют точность локализации.

6. Метрики и протокол оценки

Для query-independent clip scoring:

- MAP / Average Precision по клипам - главная метрика для ранжирования хайлайтности. Она устойчива к дисбалансу, где хороших хайлайтов меньше, чем обычных клипов.

- Precision, Recall, F1 после выбора порога - нужна для прикладного сценария, где система возвращает конечные интервалы.
- Hit@1 - полезно для preview: попал ли самый высокий score в размеченный хайлайт.
- Temporal IoU для интервалов - проверяет, насколько точно локализованы границы.

Для interval detection:

- mAP@IoU = 0.3/0.5/0.7 или mAP@[0.5:0.95] по аналогии с temporal action detection;
- Recall@K@IoU, если модель возвращает top-K интервалов.

Для TVSum/summarization sanity-check:

- F1 при бюджете summary 15% от длины видео, если используем стандартный протокол summarization;
- но его нельзя делать единственной метрикой highlight detection, потому что summary и highlight имеют разные смыслы.

Протокол:

1. Разбиение train/val/test без утечки видео-id между сплитами.
2. Подбор threshold и параметров постобработки только на val.
3. Финальная оценка на test один раз.
4. Отдельный отчёт по latency: extraction time, model inference time, memory, размер признаков.
5. Error analysis: false positives, false negatives, слишком длинные интервалы, domain shift по категориям.

7. Риски и ограничения

- Субъективность: разные пользователи считают хайлайтами разные моменты. Это видно по saliency-spread в QVHighlights и proxy-label в Mr. HiSum.
- Domain shift: vlog/news/sports/movie/cooking имеют разные паттерны интересности.
- Проблемы лицензий и доступности YouTube: сырой видеоконтент может быть удалён, а API/ToS ограничивают перераспространение.
- Баланс классов: при top-15% positive label будет сильный дисбаланс, F1 зависит от threshold.
- Границы: 2-5 секундная дискретизация ограничивает максимально достижимую точность локализации.
- Признаки: CLIP/SlowFast могут плохо ловить аудиособытия или тонкие монтажные признаки.

8. План этапа 2

1. Подготовка данных: зафиксировать основной датасет Mr. HiSum или TVSum+QVHighlights subset, скачать/подготовить метаданные, проверить лицензии, сделать train/val/test split без пересечения видео.
2. Feature extraction: выбрать частоту 1 fps или 2-секундные клипы; сохранить embeddings и mapping clip-id -> time interval.
3. Baseline metrics: реализовать random/uniform/top-k baseline и проверку MAP, F1, Hit@1, temporal IoU.
4. Clip classifier: обучить MLP/логистическую регрессию на embeddings; подобрать threshold и min-duration на val.
5. Temporal model: обучить TCN/Transformer encoder поверх последовательности clip embeddings; сравнить с MLP по quality/latency.
6. Постобработка: сглаживание score, threshold, merge соседних интервалов, удаление слишком коротких интервалов, ограничение суммарного бюджета.
7. Анализ ошибок: посмотреть FP/FN по категориям, длинам видео, длинам хайлайтов и disagreement аннотаторов.
8. Улучшения: добавить аудио или query-conditioned Moment-DETR, если базовая модель упирается в визуальные признаки.
9. Финальные артефакты: trained checkpoint, reproducible notebook, таблица метрик, demo inference script и финальный отчет.

Источники

1. Song et al. "TVSum: Summarizing Web Videos Using Titles", CVPR 2015:
https://www.cv-foundation.org/openaccess/content_cvpr_2015/papers/Song_TVSum_Summarizing_Web_2015_CVPR_paper.pdf
2. Lei et al. "QVHighlights: Detecting Moments and Highlights in Videos via Natural Language Queries", NeurIPS 2021:
<https://papers.neurips.cc/paper/2021/file/62e0973455fd26eb03e91d5741a4a3bb-Paper.pdf>
3. Moment-DETR / QVHighlights repository and data README:
https://github.com/jayleicn/moment_detr and
https://raw.githubusercontent.com/jayleicn/moment_detr/main/data/README.md
4. Sul et al. "Mr. HiSum: A Large-scale Dataset for Video Highlight Detection and Summarization", NeurIPS 2023:
https://proceedings.nips.cc/paper_files/paper/2023/file/7f880e3a325b06e3601af1384a653038-Paper-Datasets_and_Benchmarks.pdf
5. MR.HiSum repository README: <https://github.com/MRHiSum/MR.HiSum>